Reviewer Pack — Symmetric Square Irreducible Component Count Gap in Permanen…

Entry 3d93cdba3971 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 11:20:39 UTC

Symmetric Square Irreducible Component Count Gap in Permanent vs Determinant Polynomials

Entry ID: `3d93cdba3971` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-13 11:20:39 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Size for Permanent

Statement:

> For random 3-SAT instances with n variables, the number of irreducible components in the symmetric square decomposition of the corresponding polynomial (via Schur-Weyl duality) is Ω(n^2) larger than for random determinant polynomials of degree n.

Rationale (proposer's reasoning):

> Schur-Weyl decomposition reveals hidden symmetries in polynomial representations. Permanent polynomials exhibit richer symmetric structures under tensor powers, which may correlate with their inherent computational hardness. This conjecture links algebraic representation theory to circuit complexity via explicit decomposition algorithms.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a569d66117ee7459`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    num_trials = 30

    def generate_3sat_instance(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([f'x{i}', f'~x{i}']) for i in range(n)]
            random.shuffle(literals)
            clause = ' or '.join(literals)
            clauses.append(clause)
        return ' and '.join(clauses)

    def monomial_to_polynomial(monomial):
        terms = monomial.split(' and ')
        polynomial = 1
        for term in terms:
            literals = term.split(' or ')
            product = 1
            for literal in literals:
                if literal.startswith('~'):
                    product *= (1 - x[literal[2:]])
                else:
                    product *= x[literal]
            polynomial *= product
        return polynomial

    def symmetric_square_decomposition(polynomial):
        # Placeholder for actual decomposition logic
        # For simplicity, we'll just count the number of terms as a proxy
        return len(polynomial.split(' + '))

    x = {f'x{i}': random.randint(0, 1) for i in range(n)}
```

```
    permanent_like_polynomial = monomial_to_polynomial(generate_3sat_instance(n))
    determinant_like_polynomial = monomial_to_polynomial(generate_3sat_instance(n))

    permanent_shape = [(n - i) * [n - j - 1] for i in range(n) for j in range(n)]
    determinant_shape = [(i + 1) * [j + 1] for i in range(n) for j in range(n)]

    permanent_components = symmetric_square_decomposition(permanent_like_polynomial)
    determinant_components = symmetric_square_decomposition(determinant_like_polynomial)

    metric_value = permanent_components - determinant_components
    conjecture_holds = metric_value >= n**2
    counterexample = "" if conjecture_holds else f"Permanent: {permanent_components}, Determinant:

    return {
        "metric_name": "Irreducible Component Count Gap",
        "metric_value": metric_value,
        "instances_tested": num_trials,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 8)]  # First 30 prime numbers

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2f5cf482.py", line 80, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2f5cf482.py", line 52, in run_trial
    permanent_like_polynomial = monomial_to_polynomial(generate_3sat_instance(n))
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2f5cf482.py", line 40, in monomial_to_p
    product *= (1 - x[literal[2:]])
                    ~^^^^^^^^^^^^^^
KeyError: '36'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with KeyError, preventing data collection. Pre-registered support condition cannot be evaluated. | next: Debug monomial_to_polynomial() to handle literal indexing correctly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	37451
2	preregistration	ollama_remote	qwen3:8b	0	0	28170
3	novelty	ollama_remote	qwen3:8b	0	0	22799
4	novelty	ollama_remote	qwen3:8b	0	0	16615
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42391
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9628
7	judge	ollama_remote	qwen3:8b	0	0	18368

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 175422 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/3d93cdba3971.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3d93cdba3971.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3d93cdba3971.tar.gz` (if generated)